The past decade has seen the development of more platforms for distributed computing. These developments are motivated by a change in the workload behaviors from a batch-processing model to a more dynamic/streaming model where data arrives continuously through devices such as sensors, the need to build more pipelined sequences of map and reduce tasks, and provide higher-level programming abstractions.

These considerations led to the design and development of platforms that offer a higher level of abstraction compared to MapReduce. FlumeJava [11] is a Java-based programming library developed by Google for building, composing, and running data-parallel pipelines in a declarative and efficient way. It was designed to simplify the development of large-scale data processing tasks on distributed systems, particularly using MapReduce internally. Apache Beam [3] and Google Dataflow [2] generalize the ideas from FlumeJava and support both batch and stream processing. Kafka [27] is an event streaming platform designed for high-throughput, low-latency, fault-tolerant, real-time data streaming and processing. It supports horizontal scaling and offers a publish-subscribe model that can process millions of messages per second. some focus on specific domains such as machine learning [30, 1, 28], linear algebra [28], and so on. Mahout [28] is designed to build scalable, distributed machine learning algorithms on top of big data platforms such as Hadoop, Apache Spark, and Flink [10].

3.6 Summary

To summarize, the MapReduce framework is versatile to support a variety of computations involving large-scale data. The framework provides a high degree of abstraction. It offers the user a transparent mechanism to deal with many issues of distributed programming, such as scheduling and load balancing, data and task distribution, synchronization, handling faults, and so on. On the other hand, the programmer has to express the computation via only four operations: map, reduce, combiner, and partitioner. Later extensions of the MapReduce framework cater to higher-order abstractions, support for specific domains, streaming data support, and the like.

4 The Massively Parallel Computing (MPC) Model

Distributed computing has seen multiple models of computing addressing various theoretical and practical concerns. Popular among them are the LOCAL, CONGEST, CONGESTED_CLIQUE, and the MPC models.

The LOCAL model is a simple synchronous model where, in each round, each machine sends/receives messages of arbitrary size from its neighbors. This model allowed algorithm designers to study the round complexity of a distributed computation even though the message sizes were not limited. The CONGEST model is akin to the LOCAL model except for placing a bound on the message size to $O(\log n)$ bits, where n is the number of nodes in the network. Therefore, this model is closer to a practical model as it limits the communication volume across each link. The CONGESTED_CLIQUE model is useful for studying the interplay between communication and distributed algorithm round complexity. In this model, akin to the CONGEST model, the bandwidth on each communication link is in $O(\log n)$. However, the communication network allows for communication across all nodes of the network, including non-neighbors.

The three models mentioned earlier have been useful in studying distributed algorithms from a lower and upper-bound perspective. Multiple influential results exist in all three models. However, making a few aspects of the models more practical is possible. For instance, in the CONGESTED_CLIQUE model, there are no limits on the amount of space a machine has. This aspect gains importance as today's *big data* workloads correspond to very large inputs that do not fit in the space of any single machine, and one needs to distribute the input across multiple machines.

Addressing the above limitation, a model that closely resembles the cloud computing model is the Massively Parallel Computing (MPC) model. The MPC model has the following characteristics. The MPC model envisages a system with a set of nodes, each having at most W words of memory. An all-to-all communication network connects the nodes. The model assumes synchronous rounds. In each round, each node can receive up to W words in total from all other nodes, perform local computation, and send up to a total of W words to other nodes.

We measure the performance of an algorithm in the MPC model by the number of communication rounds needed by the algorithm. This measure ignores the time taken by local computation and seems unnatural. We justify this by the widespread belief that communication costs usually dominate the overall cost in big data computations. See also Reed and Dongarra[32].

If the input has a size of N, this suggests that the number of machines M is such that $M \times W \in \Theta(N)$. Thus, a key characteristic of the MPC model is that both the memory bound per machine, W, and the number of machines, M, used are assumed to be strongly sublinear in terms of the input size N, i.e., bounded by $O(N^\delta)$ for a positive δ.

The above indicates that, unlike other distributed computing models such as the CONGESTED_CLIQUE model, the number of machines needed in the MPC model grows sub-linear as the input size grows. Such sublinear growth makes the model more realistic than other distributed computing models. The dependence on the number of machines also allows the model to be elastic. It resembles one of the maxims of cloud computing that will enable them to adjust the resources on demand. On the other hand, unlike models such as the BSP [33] and the k-machine model [21], keeping the number of machines dependent on the size of the input may also be seen as a weakness of the model.

4.1 A First MPC Algorithm

Let us consider the problem of sorting N numbers and see how we can design an MPC algorithm for this problem. Initially, we imagine that the N numbers are stored so that each machine has W numbers. The size of the input is N. In the following, we consider the case where $W = N^\delta$ for a constant $\delta > 0$. This limitation precludes the possibility of storing all the numbers in any single machine. The output is the rank of each element in the sorted order. Other output models are possible, e.g., requiring that the first machine has the first W numbers according to rank, the second machine the next W numbers, and so on. We note that it is possible to rearrange the elements at the end to meet such output requirements.

The algorithm we design resembles the quicksort algorithm [12] with changes to account for the MPC model. The basic idea is to use multiple pivots so that there are $O(W)$ elements between any two successive pivots. Recall that $W = N^\delta$ and we assume for simplicity that $\delta \geq 1/2$. In the case $\delta < 1/2$, the analysis and the algorithm are to be modified slightly. With such a pivoting, it is possible to sort each such set of elements within two consecutive pivots locally, i.e., in a single machine. The details follow.

For $i = 1, 2, \cdots$, let N_i refer to the set of input numbers stored at machine M_i. We can use $W = |N_i|$, since each machine has a space for W numbers. The crux of the algorithm is to choose a good set of pivots. To this end, each machine marks each element in N_i as a pivot with a probability p, independently. We will specify the exact value of p during our analysis. Thus, each machine on expectation marks $W \cdot p$ pivots.

All machines send their choice of pivots to all other machines. Each machine then computes the local rank of each pivot. The local rank of a pivot q in a machine M_i, denoted $\mathrm{Rank}(q, i)$, is the number of elements in M_i smaller than the q. Each machine then sends the local ranks of all the pivots to a designated machine M.

Machine M uses these local ranks to identify a good set of $N \cdot p$ pivots. Machine M chooses one pivot for every N^δ elements of the input. The analysis below shows that with high probability, every N^δ portion of the (sorted) input has at least one pivot. Let the chosen pivots be $Q := \{q_1, q_2, \cdots, q_{N^{1-\delta}}\}$. Machine M then sends Q to all other machines.

Each machine now partitions its input into $|Q| + 1$ buckets, with bucket j having elements that have a value between the $(j-1)^{\text{th}}$ and j^{th} pivot (of Q).

In a final round of communication, machines exchange elements such that all elements x currently at machine M_i with a value between q_{j-1} and q_j are sent to machine M_j. (Use $q_0 = -\infty$ and $q_{|Q|+1} = \infty$).

At this point, machine M_j has a set of input numbers with a value between q_{j-} and q_j. This machine can then sort these numbers locally and assign ranks to these numbers starting from the global rank of pivot $q_{j-1} + 1$ to the global rank of q_j.

Algorithm 1 shows the pseudocode of the above approach.

Algorithm 1 Quicksort in MPC.

1: Each machine M_i marks each element of S_i as a pivot with probability p;
2: Each M_i sends all the pivots chosen by M_i to all other machines
3: Each machine M_i calculates the local rank of each pivot, $\mathrm{LocalRank(q, S_i)}$.
4: Each machine M_i sends its $\mathrm{LocalRank}$ to a designated machine M, say M_1
5: Machine M_1 uses the arrays to designate Np pivots and their global ranks, $\mathrm{Rank(q, S)}$ and communicates the pivots and Rank to all machines
6: Each machine M_i sends the elements with a value between q_{j-1} and q_j to machine M_j
7: Each machine M_i sorts S_i locally and assigns a rank

4.1.1 Analysis

We start the analysis with the following lemma that indicates that every segment of length $N^{1-\delta}$ has at least one element marked as a pivot.

Lemma 1. *With high probability, some machine marks at least one element in every segment of $N^{1-\delta}$ ranks of the input A.*

Proof. The probability that no machine marks any of the elements in a given segment of $N^{1-\delta}$ ranks of the input array is $(1-p)^{N^{1-\delta}}$. This probability is at most $e^{-pN^{1-\delta}}$. Setting $p = \frac{4W \log N}{N}$, the above probability is at most $e^{-4 \log N} = 1/N^4$. There are at most N segments of length $N^{1-\delta}$. We now use the union bound over all segments to conclude that with probability at least $1 - 1/N^3$, in every segment of $N^{1-\delta}$, there exists at least one element marked as a pivot. □

Now, let us analyze the number of communication rounds the algorithm needs. An important component of this analysis is to check the communication volume since, in each round, each machine can send and receive up to $\tilde{O}(W)$ words.

Each machine selects $O(W \cdot p)$ pivots and hence is sending/receiving $O(W \cdot p)$ pivots from each of the other machines, on expectation. The total number of words that a machine receives is in $O(\frac{N}{W} \cdot W \cdot p) = O(N \cdot p)$. With $p = \frac{4W \log N}{N}$, the communication volume at each machine is in $\tilde{O}(W)$. Further, the last communication step of rearranging elements also has a communication volume of $O(W)$ at each machine. Following the above analysis, we note that the algorithm to sort N elements runs in $O(1)$ communication rounds in the MPC model.

4.1.2 Observations

The above algorithm and its analysis show how the number of machines and the space per machine trade off against each other. Notice that through the execution of the algorithm, no single machine needs to look at every element of the input. The ability of the algorithm to work with incomplete inputs is a good lesson in

designing algorithms in the MPC model, especially when the entire input does not fit in any single machine. So, the algorithm has to find mechanisms to ensure that machines can execute the required function without routing the entire input to every machine.

The algorithm derives its efficiency from the fact that it is possible to identify a good set of pivots using simple mechanisms. There exist other ways to identify such a good set of pivots. Using similar ideas, one can also design MPC algorithms for other semi-numerical computations, such as frequency estimates and prefix sum that run in $O(1)$ rounds in the MPC model. It is interesting to note that MapReduce has a sort functionality that sorts keys from all mappers.

4.2 MPC Algorithms for Graphs

Graphs are a popular mechanism for representing relationships across entities. For this reason, algorithms for graph problems, including various distributed computing models, have been popular. Algorithms for graph problems in the MPC model have recently been a popular research topic. Such studies have also thrown interesting insights into how the space per machine affects the design of algorithms.

In particular, we note that when focusing on graph inputs with n nodes and m edges, the input size is $N = \tilde{O}(m + n)$, where m is the number of edges and n is the number of nodes of the input graph. Thus, as per the description of the MPC model earlier, $W \in \tilde{O}(m + n)^{\epsilon}$ for $0 < \epsilon < 1$. In the above, the notation $\tilde{O}(f(n))$ corresponds to functions in $O(f(n) \log^{O(1)} f(n))$. In other words, the $\tilde{O}$ notation hides multiplicative polylog factors.

To proceed further, we seek an appropriate range of values for W and the interplay of these ranges to the performance of distributed algorithms. In this direction, we study three different ranges for W. Recall that n refers to the number of nodes of the input graph.

- Strongly super-linear Memory Regime: In this case, we let $W = O(n^{1+\epsilon})$ for $0 < \epsilon < 1$. With such a large space at each machine, it is important to note that for the model to be realistic, we need the graph to be highly dense, i.e., $m \gg W \gg n$, such that W is strongly sub-linear in m. Even as the input graph is dense, the fact that each machine has $O(n^{1+\epsilon})$ local memory makes this model quite powerful. For example, in this model, problems such as a minimum spanning tree, MIS, and 2-approximate minimum vertex cover all have $O(1)$-round algorithms [18, 20, 24].

- Near-linear Memory Regime: In this regime, we let $W = \tilde{O}(n)$. Therefore, each machine has space to store $O(n)$ information. Due to this space limit, most graph algorithms need more rounds than in the strongly super-linear regime. There exist very few graph problems with an $O(1)$ round algorithm: see, e.g., vertex coloring [4] and MST [15].

- Strongly sub-linear Memory Regime: In this setting, we let $W = O(n^{\epsilon})$ for $0 < \epsilon < 1$. Therefore, each machine has very limited space and cannot even store information about every vertex of the graph. Due to this limited space per machine, most graph algorithms tend to have a higher round complexity. In fact, from a lower bounds perspective, it is *conjectured* that the problem of distinguishing if the input graph is a single cycle vs. two disjoint cycles of length $n/2$ requires $\Omega(\log n)$ rounds [16]. However, sublogarithmic-round algorithms exist for several graph problems [17, 22].

To showcase the typical techniques used in designing algorithms in the three MPC regimes, we consider the problem of obtaining a minimum spanning tree (MST) of a given graph.

4.2.1 Strongly super-linear Memory Regime

One popular technique in the strongly super-linear regime is *f*iltering [24]. Let $m = n^{1+c}$ for a constant $0 < c < 1$. Notice that $c > \epsilon$ as otherwise, the problem is easy to solve. Consider a partitioning of the edges

of G as $E_1, E_2, \cdots E_r$ such that $|E_i| = n^{1+\epsilon}$. Let $T_i = \text{MST}(E_i)$. Subsequently, we recursively compute the MST of $\cup_i T_i$ and return the resulting MST.

Algorithm 2 super-linear MST algorithm.

```
c = 4, t = n^{1+ε}/c
while |E| > t do
    Partition E into E_1, E_2, ··· with |E_i| = t
    Rearrange E such that for i ≥ 1, machine M_i stores E_i.
    for each i in parallel do
        G_i = (V, E_i)
        Compute T_i = MSF(E_i), the minimum spanning forest using edges in E_i (locally)
    end for
    E = ∪_i T_i
end while                    ▷ On loop exit, |E| is small enough to fit in a single machine.
return MST(G = (V, E))
```

Algorithm 2 captures the above as pseudocode. The following lemmata show the correctness of the algorithm and the number of rounds required.

Lemma 2. *Let $G = (V, E)$ be a weighted graph. Consider a subset F of E and let G' be subgraph of G spanned by the edges in F. If an edge $e \in F$ is not in the MST (or the minimum spanning forest (MSF)) of G', then e is not in the MST of G.*

Proof. The proof uses the cycle property of the MST. The cycle property states that any edge not in the MST is the heaviest edge on some cycle in G.

Suppose that an edge $e \in F$ is not included in the MSF of G'. According to the cycle property, e is the heaviest edge on some cycle, say C in G'. This cycle is preserved in G too since G' is a subgraph of G. This suggests that the edge e would be the heaviest edge on C. Hence, e cannot be in the MST of G either. □

Lemma 3. *The algorithm needs $O(1/\epsilon)$ rounds.*

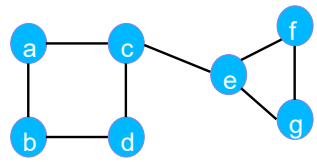
Proof. The number of rounds the algorithm needs can be analyzed as follows. Notice that the number of partition is $m/n^{1+\epsilon} = n^{c-\epsilon}$ as $m = n^{1+c}$. The number of machines is $\frac{m}{n^{1+\epsilon}} = \frac{n^{1+c}}{1+\epsilon} = n^{c-\epsilon}$. So, each recursive call would result in the number of edges to at most $n^{c-\epsilon} \times n = n^{1+c-\epsilon}$. Therefore, it holds that after $\frac{c}{\epsilon} - 1$ recursive calls, the number of remaining edges would be at most $n^{1+\epsilon}$. This edge count is small enough to fit in a single machine. We can then solve the problem by keeping all the remaining edges in one machine. Hence, the algorithm runs in $O(c/\epsilon)$ rounds. □

Such $O(1/\epsilon)$ algorithms also work for other graph problems in this regime (cf. [24]). Observe that algorithms can benefit from each machine having space in $O(n^{1+\epsilon})$.

We also notice that the pseudocode in Algorithm 2 translates to a MapReduce implementation.

4.2.2 Near Linear Memory Regime

In the near-linear memory regime, one of the powerful techniques to design graph algorithms is *graph sketching*. Sketching is a useful technique in the context of streaming algorithms. This technique found its use in distributed computing through the work of Pandurangan el al. [29]. Using the idea of sketching, we show that the MST of a graph can be obtained in $O(1)$ rounds in this regime. The algorithm is more involved than the corresponding algorithm in the super-linear memory regime and explores the connections between MSTs and connected components of a graph that we dwell in the following.



Edge	End points	ID
e1	a, c	000010
e2	a, b	000001
e3	b, d	001011
e4	c,d	010011
e5	e, f	100101
e6	f, g	101110
e7	e, g	100110
e8	c, e	010100

Node	Edges	XOR_u
a	e1, e2	000011
b	e2, e3	001010
c	e1, e4, e8	000101
d	e3, e4	011000
e	e5, e7, e8	100111
f	e5, e6	001011
g	e6, e7	111000

Figure 1: For the graph shown in the top part of the figure, the tables show the edge ids and XOR_u for each vertex u. The figure assumes that the numbers for the nodes a through g are 000 through 111, in alphabetic order.

Let us consider the edges of G in increasing order of weights. Let us label them $e_1, e_2, \cdots, e_m$ in sorted order. Note that we can use the algorithm approach from Section 4.1 and obtain the sorted order of edges in $O(1)$ rounds. The following observation holds. This observation is part of the famous algorithm of Kruskal [12].

Observation 1. *For any $i \geq 1$, edge e_i is in the MST of G if and only if the endpoints of e_i are in different connected components in the graph defined by the edges $\{e_1, e_2, \cdots, e_{i-1}\}$.*

An immediate application of Observation 1 implies that one needs to use an algorithm for connected components m times. However, by grouping edges, one can leverage the power of the MPC model and make fewer calls, i.e, m/n calls, to the connected components algorithm.

The idea is to keep $O(n)$ edges in each machine. So, we plan to group edges into groups of size $O(n)$ each. To this end, let E_i refer to the ith group of m/n edges in the sorted order of weights. In other words, $E_i = \{e_{(i-1)\cdot n+1}, e_{(i-1)\cdot n+2}, \cdots, e_{i\cdot n}\}$. Let F_i refer to the union of the first i sets of E_i, i.e., $F_i = \cup_{j=1}^{i} E_i$. Let H_i refer to the MST of the graph defined by edges in F_i. (We set F_0 as the empty set.)

Notice that $|E_i| \in O(n)$ and also that $|H_i| \in O(n)$. So, a single machine can store E_i and H_i. Further, according to Observation 1, for any edge $e \in E_i$ with endpoints u and v to be in H_i, u and v must be in different components in H_{i-1}. Therefore, we can identify the edges in the MST using the connected components algorithm as a subroutine.

We will now design an efficient algorithm for the connected components problem in the near-linear memory regime using graph sketches. For simplicity, let us assume that for any set $U \subseteq V$ of vertices, the cut between U and $\overline{U} = V \setminus U$ has a single edge. We want to use the idea of a graph sketch to identify the edge that crosses the cut $(U, \overline{U})$. Note that a vertex u may have $\Theta(n)$ neighbors, so it is not easy to even list the neighbors of a vertex in a compact manner. The sketch idea works as follows.

Consider writing each edge uv as the concatenation of the identifiers of u and v. In other words, if $id(u)$ refers to the (unique) identifier of vertex u, the $id(e) = u \cdot v$ for an edge e. Each vertex u locally computes $XOR_u = \oplus_{w \in N(u)} id(uw)$. Figure 1 shows an example.

Suppose we know XOR_u for each vertex u. Observe that an edge that has both endpoints in U (or $\overline{U}$) is included twice in the XOR computation of vertices in U (and $\overline{U}$); once in the calculation from each of its endpoints. On the other hand, for the cut edge $e = uv$ that crosses the cut $(U, \overline{U})$, u and v are included in the calculation of XOR_u and $XORv$ only once. Therefore, if we compute $XOR_U = \oplus_{u \in U} XOR_u$, then we

get the id of the edge that crosses the cut $(U, \overline{U})$. From Figure 1, we note that $XOR_U = XOR_a \oplus XOR_b \oplus XOR_c \oplus XOR_d = 000011 \oplus 001010 \oplus 000101 \oplus 011000 = 010100$, which is the id of the edge ce.

While the above idea is helpful, it does not extend seamlessly if the cut $(U, \overline{U})$ has > 1 edges. In this case, all the cut edges appear only once in XOR_U. The resulting XOR_A may be the id of no edge or the id of some edge that is not a cut edge!

We first see how to salvage the idea to the case where the cut $(U, \overline{U})$ has > 1 edges. In this case, let us imagine that we know an estimate k' for the size of the cut k such that $k/2 < k < k'$. For such an estimate k', we mark each edge with probability $1/k'$. Let us now calculate the probability of the event E that exactly one of the k cut edges is marked.

$$\begin{aligned}
\Pr(E) &= \binom{k}{1} \cdot \frac{1}{k'} \cdot \left(1 - \frac{1}{k'}\right)^{k-1} \\
&\geq \frac{k'}{2} \cdot \frac{1}{k'} \cdot \left(1 - \frac{1}{k'}\right)^{k-1} \\
&\geq \frac{1}{2} \cdot \frac{1}{4} \text{ as } 1 - x \geq 4^{-x} \text{ for } x \in R \\
&= 1/8
\end{aligned}$$

The above calculations show that a fixed cut edge is marked, and no other cut edges are marked at least a constant. At this point, one can use the idea from the single-cut edge case to identify the cut edge.

A few minor details need to be addressed, which we discuss in the following.

- First is the case of distinguishing that there is exactly one cut edge versus the case of multiple cut edges. Such a distinction is needed since it is likely that if E does not occur and we use the sketching idea, then XOR_U may equal the id of some edge of the graph. Otherwise, the algorithm may produce an erroneous conclusion with respect to the cut $(U, \overline{U})$.

  A solution to this problem is to redefine the edge id as a random string of $O(\log n)$ bits instead of using the concatenation of the ids of the endpoint of the edge. This randomness helps as follows. Suppose we assign a uniformly chosen random string of $\log n$ bits as an id to each edge. Since the XOR of bit strings chosen uniformly at random is also a bit string that appears to be chosen uniformly at random, we have the following:

$$\begin{aligned}
\Pr(XOR_U = id(e) \text{ for some edge e} | \neg E) &\leq \textstyle\sum_{e \in E(G)} \Pr(XOR_U = id(e) | \neg E) \\
&\leq \binom{n}{2} \cdot \left(\frac{1}{2}\right)^{8 \log n} \\
&\leq \frac{1}{n^6}
\end{aligned}$$

  This random assignment of edge ids allows us to distinguish between the outcome E and $\neg E$.

- Second, since vertices mark their edges, each edge gets marked twice: once by each endpoint. These marks may not be identical. This problem can be solved by having nodes use shared randomness. Another way to solve this problem is for each edge, let the endpoint with the smaller ID do the marking and inform the outcome of the marking to the other endpoint. This may require an additional $O(1)$ rounds of communication[4].

- Third, we do not know the estimate k'. In the MPC model, we try all possible k'. Since $k \in O(n^2)$, we can work with $O(\log n)$ estimates of $k' = 1, 2, 4, 8, \cdots, n^2$. One of these will happen to be the *correct* estimate of k'. Further, executing all possible estimates can happen in parallel in the MPC model. Even as doing so increases the total work, the number of communication rounds is unaffected due to trying all these multiple possibilities.

[4]With thanks to an anonymous reviewer who mentioned this possible solution.

In the MPC model, the algorithm for MST in the near-linear regime uses $O(\log n)$ sketches per vertex. These together require $O(\log^4 n)$ space per vertex. Thus, the sketches of all vertices fit in a single machine with $\tilde{O}(n)$ space.

Each machine M_i computes the connected components of the edges in $\cup_{j=1}^{i} E_j$. Once the sketches are shared across all machines, each machine can then identify the connected components locally. These connected components with $O(n)$ edges constitute the graphs H_i for each i.

Using this as a sub-routine, we also conclude that the MST problem can be solved in $O(1)$ rounds in the near-linear MPC regime.

4.2.3 Strongly sub-linear Memory Regime

We now move to the sub-linear memory regime, where it is not possible to use the sketching idea due to the limited space on each machine. On the other hand, a careful simulation of Boruvka's algorithm [8] can be made to run in the sub-linear MPC regime to obtain an MST in $O(\log n)$ rounds. The details follow.

We first briefly introduce Boruvka's algorithm and then describe how to implement it in the sub-linear memory MPC regime. Let $G = (V, E, W)$ be a weighted undirected graph, and T be the MST of G. For simplicity, we assume that T is unique. Boruvka's algorithm relies on the following observation, which is often mentioned as the cut property of the minimum spanning tree.

Observation 2. *Let $G = (V, E, W)$ be a weighted graph and S be a subset of V with $\overline{S} = V \setminus S$. Denote by $(S, \overline{S})$ the set of edges with exactly one endpoint in S and the other endpoint in $\overline{S}$. Note that $(S, \overline{S})$ also forms a cut in G. Given any cut $(S, \overline{S})$, the edge of the least weight crossing the cut must be in the minimum spanning tree of G.*

Now, consider the set F of edges obtained by marking all the edges of every minimum weighted edge incident to every vertex v. We note that the set F of edges is acyclic and a subset of T. The algorithm of Boruvka extends these ideas. Initially, T is empty, and every vertex sets itself to be its representative vertex. We use the array R to store the representative vertex. In the first phase, each vertex v of G identifies the edge incident to v and has the smallest weight among such edges. We add this set of edges, F, to T. In each connected component of T, we let the vertex with the smallest number be the representative of all the vertices in that connected component. In subsequent phases, for each connected component C of T, we find the edge of the smallest weight with one endpoint in C and another endpoint not in C. Identifying such edges requires using the representative vertex. Such an edge uv will have $R[u] \neq R[v]$. We now appeal to Observation 2 and add to T all such edges that have the minimum weight among edges that go across connected components of T. The algorithm runs until T has $n - 1$ edges.

Using standard techniques, one can show that the algorithm requires $O(m)$ time for each phase, where $m := |E(G)|$. Notice that in each phase, the number of vertices that have the same representative vertex will increase at least by a factor of 2. Therefore, the algorithm needs $O(\log n)$ phases to obtain T. Put together, the algorithm runs in $O(m \log n)$ time.

MPC Implementation We first observe that each phase of the algorithm runs sequentially. So, we focus on how to run each phase in $O(1)$ rounds of MPC. This approach results in $O(\log n)$ rounds for the entire algorithm. Each phase has two main steps: finding the crossing edge with the minimum weight and updating the representative pointer.

Notice that a naive implementation where each machine wants to inform all other machines of the edges to include in the MST results in a lot of communication volume. As each machine is limited to sending/receiving no more than $O(n^\epsilon)$ Bytes, this will result in each phase requiring more than $O(1)$ rounds. Another complication is that the set of edges to add in this phase may result in long chains of changes to

representative pointers, increasing the time needed per phase. So, we employ the following structure and a randomized strategy [5].

Consider arranging the set of machines across $\ell = O(\log_W m)$ levels so that we have $4m/W$ machines at each level. For now, assume that the machines are all distinct. We number the levels starting with level 1 at the bottom and level ℓ at the top. Further, we fill level 1 with the $4m/W$ machines that store the input graph.

Let us associate a fragment id with every vertex. Initially, each vertex has its own id as its fragment id. We say that a machine M on a level i is working for a fragment ID f if it participates in the computation for the fragment with ID f. On level 1, a machine M is working for fragment f if and only if it initially stores an edge u, v such that either u or v is in fragment f. At level 1, we say that machine M works for fragment id f if M has an edge uv belonging to fragment f.

Across two consecutive levels i and $i+1$, with $i \geq 1$, we also define a parent-child relationship across machines as follows.

In each level $i \geq 2$, for each fragment ID f, the set of machines responsible for fragment ID f on level $i \geq 2$ are chosen as a uniformly random subset of size $m/(S/4)^{i+1}$ of the $4m/S$ machines of level i. On the last level, numbered ℓ, we choose one machine as responsible for a given fragment id f.

Based on the above set up, additionally, for each fragment id f and each level i, for $1 < i < \ell$, each machine M on level i that works for fragment id f is assigned a parent machine M' in level $i+1$. Machine M chooses a machine M' as its parent with respect to fragment id f from the subset of machines that are responsible for fragment id f in level $i+1$.

On level $i \geq 2$, a machine M is responsible for fragment F if a machine M exists on level $i-1$ such that M is the parent with respect to fragment F of level $i-1$. We define the in-degree of a machine M' on level $i > 1$ is the number of pairs (M, f) such that M' is the parent of machine M with respect to fragment F on level $i-1$.

In the following, we show that the in-degree of a machine in a level $i > 1$ is at most $W/2$. Note that the lemma assumes that $\alpha > 1/2$. For a different α, the constants in the calculations change. Figure 2 shows an example of the structure with machines corresponding to fragment f marked accordingly.

Lemma 4. *Let M be a machine in some level i for $1 < i < \ell$. Then, the in-degree of M is in $O(W)$.*

Proof. Consider a fragment id f and a level $i > 1$. Note that a subset of $M/(W/4)^{i+1}$ machines in level i are chosen for fragment id f. So, the probability that machine M is chosen for fragment id x is $\frac{m/(W/4)^{i+1}}{m/(W/4)}$ which is $\left(\frac{4}{W}\right)^i$.

Under the above condition, the expected number of children for M is the ratio of the number of machines in level $i-1$ working for fragment F over the number of machines in level i working for fragment F. This ratio is $\frac{m}{(W/4)^i} / \frac{m}{(W/4)^{i+1}} = W/4$.

The unconditional expected number of children of M with respect to fragment id f is the product of the above two probabilities and is $\left(\frac{4}{W}\right)^i \times \frac{W}{4} = \left(\frac{4}{W}\right)^{i-1}$.

Using these, we bound the expected in-degree of machine M as the product of the number of fragments and the expected number of children with respect to a given fragment. There are at most n fragment ids. Thus, the expected in-degree of machine M is at most $n \times \left(\frac{4}{W}\right)^{i-2}$. With $W = n^\epsilon$, we can simplify the product as at most $W^{1/\epsilon} \cdot \left(\frac{4}{W}\right)^{i-1}$. Now, we use that $i \leq \ell$ and $\ell = \log_{W/4} m = \frac{\log_W m}{1 - \log_W 4} \leq 2 \log_W(n^2) = 4 \log_{n^\epsilon} n = 4/\epsilon$. □

From the above lemma, we can also note that if the structure is viewed per fragment id, then the structure resembles a tree with a degree $W/2$ and depth $4/\epsilon$. Figure 3 shows an example. This tree structure helps

[5] The following is adapted from a lecture notes by Prof. Kuhn `https://ac.informatik.uni-freiburg.de/teaching/ss_20/distributed_systems/solutions/solution_12.pdf`

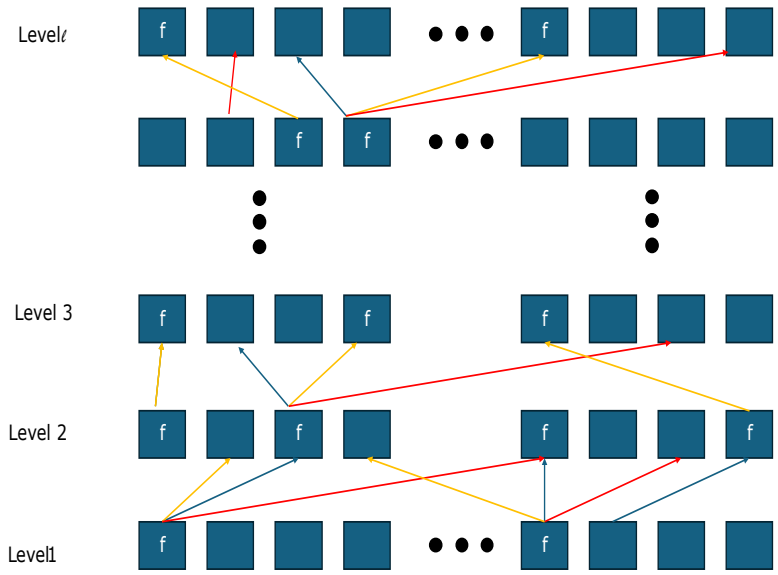


Figure 2: Machines arranged across the ℓ levels. Some machines work for fragment id f as marked in the figure. Each colored edge from a machine in one level to a machine in the next higher level indicates the parent-child relationships. Edges of the orange color correspond to fragment id f.

reduce the communication volume by not requiring an all-to-all communication to inform the edges to be added to the MST in this phase and propagate the changes to the representative pointers. The details follow.

For each fragment id f, each machine M on level 1 sends the minimum weight edge with exactly one endpoint in fragment id f that it stores to its parent. Continuing this level-by-level traversal, each machine on a level $i \geq 2$ waits to receive information from its children in level $i-1$. They can then send the required information to their parent with respect to each fragment id f. Eventually, this information is collected at the last level ℓ.

Let uv be the minimum weight edge that the machine on the top level receives for a fragment id f, with u in f and v in another fragment id h. Imagine that all the fragments are colored blue and red with equal probability. While we omit the details, machines can color fragments consistently and in parallel via shared randomness on different machines. At the last level, ℓ, for an edge uv to be added, the machine checks whether u is in a red-colored fragment id f and v is in a blue-colored fragment id h. Only then, the machine in level ℓ adds edge uv to the MST and initiates the change of representative from fragment id f to that of fragment id h. This machine then broadcasts this change in the representative to its children with respect to fragment id f. Subsequently, for each fragment, each machine broadcasts this information to its children until this information reaches machines in level 1.

As the depth of the tree is $\ell \in O(4/\epsilon)$, the entire process runs in $O(1/\epsilon)$ rounds of MPC. The communication volume is bound by the in-degree of any machine, which is shown to be in $O(W)$ from Lemma 4.

From the above, we infer that each phase of the Boruvka algorithm runs in $O(1)$ rounds in the sub-linear MPC model. The total round complexity is $O(\log n)$ phases as the algorithm needs $O(\log n)$.

Remark 1. *In the earlier sections, we focused on the MST problem. We noticed that the range of techniques required and the nature of the solution differ significantly across the three memory regimes. In particular, the case of the sub-linear memory regime requires one to use more sophisticated techniques. The MST algorithm in that regime is also randomized compared to the deterministic algorithms for the other two models.*

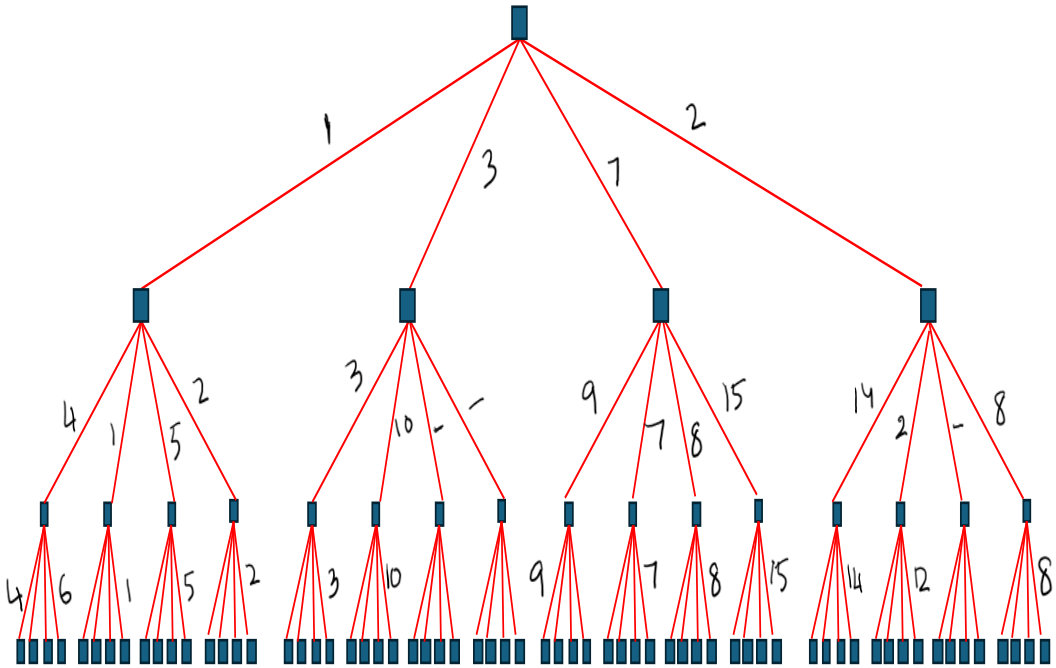


Figure 3: The tree of machines formed by the parent-child relations across the ℓ levels. This tree helps identify the minimum weight outgoing edge for a given fragment in $O(1)$ rounds. The numbers on the edges indicate the weight of the minimum weight outgoing edge sent by children to its parent. A "_" on an edge indicates that the corresponding child machine does not have any such edge for this fragment.

Remark 2. *While our chapter focuses on the minimum spanning tree algorithms in the three different memory regimes, MPC algorithms in the three memory regimes exist for various graph problems. Popular among them are maximal independent sets, connectivity, maximal matching, coloring, ruling sets, and the like. There is a vast literature on these problems and their corresponding MPC algorithms. For a more detailed review of such literature, we refer the reader to the book by Ghaffari [15].*

5 The Bridge between MPC and MapReduce

The past two sections briefly described the popular distributed programming platform MapReduce and a distributed algorithm model, MPC, along with examples. In this section, we discuss how to view the algorithm model and the programming platform as a way to bridge the theory and practice of distributed systems by comparing and contrasting them.

The programming platform and the MPC model diverge in two aspects. Firstly, the MapReduce system does not explicitly limit the space per machine. Similarly, the programming framework does not have a mechanism to count the number of rounds of communication needed. The MapReduce framework, instead, is typically studied for the run time the program takes on a given input with a specified number of mapper and reducer machines.

The MPC model, in contrast, enforces a discipline on algorithm designers to design algorithms that respect the space limit on any individual machine. By doing so, we expect that the algorithm and the corresponding implementation in a MapReduce framework continue to work on varying input sizes.

Lattanzi et al. [24] suggest that one can analyze MapReduce programs for the number of rounds by considering one Map function call followed by one Reduce function call as one round. In the limited context of graph algorithms, Lattanzi et al. [24] argue that it is possible to implement every algorithm designed in

the super-linear memory MPC model as $O(1)$ round algorithms in the MapReduce model. Here, $O(1)$ rounds refer to the number of map and reduce rounds that the program in the MapReduce framework needs. However, their work does not comment on algorithms for problems from domains other than graphs and memory regimes beyond the super-linear regime.

Another important aspect to note is that the MapReduce framework requires programmers to express every computation as a sequence of map and reduce functions, each of which works on (key, value) pairs. It is not possible to express *every* computation through map and reduce functions. Recent platforms proposed after MapReduce such as [10, 11, 2] may alleviate some of these concerns. On the other hand, we express MPC algorithms using general-purpose statements, and it is possible to write every such MPC algorithm as a suitable program in a distributed message-passing system. However, as the MapReduce framework and the frameworks proposed subsequently [11, 27, 2] offer useful mechanisms that ease the job of writing distributed programs, it is a good model to study from a practical standpoint.

Some algorithms designed in the MPC model suffer from work bloat. An example of such an algorithm is the 2-ruling set algorithm of Kothapalli et al. [22] in the sublinear memory regime. In such cases, the amount of work done by all the machines put together to execute an MPC algorithm may be asymptotically larger than the work that an optimal algorithm for the same problem requires. This situation arises as MPC algorithms optimize the communication round complexity.

The above discussion indicates that while it is possible to think of MPC algorithms and MapReduce programs as the two sides of the theory and practice of distributed systems, a few gaps still exist. Ideally, practical frameworks should include provisions for limits on space and network bandwidth. Similarly, algorithmic models such as the MPC should look at optimizing the round complexity and the overall work done by the algorithm. Another approach would be to arrive at a combination of parameters that help bridge the theory and practice of distributed systems.

References

[1] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., ET AL. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (2016), pp. 265–283.

[2] AKIDAU, T., BRADSHAW, R., CHAMBERS, C., CHERNYAK, S., FERNÁNDEZ-MOCTEZUMA, R. J., LAX, R., MCVEETY, S., MILLS, D., PERRY, F., SCHMIDT, E., AND WHITTLE, S. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment 8* (2015), 1792–1803.

[3] APACHE SOFTWARE FOUNDATION. Apache beam (version 2.61.0). https://beam.apache.org, 2025.

[4] ASSADI, S., CHEN, Y., AND KHANNA, S. Sublinear algorithms for (δ + 1) vertex coloring. In *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)* (2019), pp. 767–786.

[5] BAHMANI, B., MOSELEY, B., VATTANI, A., KUMAR, R., AND VASSILVITSKII, S. Scalable k-means++. *Proc. VLDB Endow. 5*, 7 (2012), 622–633.

[6] BATENI, M., BEHNEZHAD, S., DERAKHSHAN, M., HAJIAGHAYI, M., KIVERIS, R., LATTANZI, S., AND MIRROKNI, V. S. Affinity clustering: Hierarchical clustering at scale. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA* (2017), pp. 6864–6874.

[7] BHASKARA, A., AND WIJEWARDENA, M. Distributed clustering via LSH based data partitioning. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018* (2018), J. G. Dy and A. Krause, Eds., vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 569–578.

[8] BORVVKA, O. O jistem problemu minimalním [about a certain minimal problem]. *Prace Mor. Prirodoved. Spol. V Brne III (in Czech and German) 3* (1926), 37–58.

[9] BRADY, H. E. The challenge of big data and data science. *Annual Review of Political Science 22*, 1 (2019), 297–323.

[10] CARBONE, P., KATSIFODIMOS, A., EWEN, S., MARKL, V., HARIDI, S., AND TZOUMAS, K. Apache flink™: Stream and batch processing in a single engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering 36*, 4 (2015), 28–38.

[11] CHAMBERS, C., RANIWALA, A., PERRY, F., ADAMS, S., HENRY, R. R., BRADSHAW, R., AND WEIZENBAUM, N. Flumejava: easy, efficient data-parallel pipelines. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation* (2010), PLDI '10, p. 363–375.

[12] CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., AND STEIN, C. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.

[13] DEAN, J., AND GHEMAWAT, S. Mapreduce: a flexible data processing tool. *Commun. ACM 53*, 1 (2010), 72–77.

[14] FREDERICK P.BROOKS JR. No silver bullet - essence and accidents of software engineering. *Computer 20*, 4 (1987), 10–19.

[15] GHAFFARI, M. Massively parallel algorithms. `http://people.csail.mit.edu/ghaffari/MPA19/Notes/MPA.pdf`, 2019.

[16] GHAFFARI, M., KUHN, F., AND UITTO, J. Conditional hardness results for massively parallel computation from distributed lower bounds. In *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019* (2019), D. Zuckerman, Ed., IEEE Computer Society, pp. 1650–1663.

[17] GHAFFARI, M., AND UITTO, J. Sparsifying distributed algorithms with ramifications in massively parallel computation and centralized local computation. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019* (2019), T. M. Chan, Ed., SIAM, pp. 1636–1653.

[18] HARVEY, N. J. A., LIAW, C., AND LIU, P. Greedy and local ratio algorithms in the mapreduce model. In *Proceedings of the ACM Symposium on Parallelism in Algorithms and Architectures, SPAA* (2018), p. 43–52.

[19] IM, S., KUMAR, R., LATTANZI, S., MOSELEY, B., AND VASSILVITSKII, S. *Massively Parallel Computation: Algorithms and Applications*, vol. 5(4). NOW, 2023, pp. 340–417.

[20] KARLOFF, H., SURI, S., AND VASSILVITSKII, S. A model of computation for mapreduce. In *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)* (2010), pp. 938–948.

[21] KLAUCK, H., NANONGKAI, D., PANDURANGAN, G., AND ROBINSON, P. Distributed computation of large-scale graph problems. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA* (2015), P. Indyk, Ed., SIAM, pp. 391–410.

[22] KOTHAPALLI, K., PAI, S., AND PEMMARAJU, S. V. Sample-and-gather: Fast ruling set algorithms in the low-memory MPC model. In *40th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2020, December 14-18, 2020, BITS Pilani, K K Birla Goa Campus, Goa, India (Virtual Conference)* (2020), N. Saxena and S. Simon, Eds., vol. 182 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 28:1–28:18.

[23] KUMAR, R., MOSELEY, B., VASSILVITSKII, S., AND VATTANI, A. Fast greedy algorithms in mapreduce and streaming. *ACM Trans. Parallel Comput. 2*, 3 (2015), 14:1–14:22.

[24] LATTANZI, S., MOSELEY, B., SURI, S., AND VASSILVITSKII, S. Filtering: a method for solving graph problems in mapreduce. In *SPAA 2011: Proceedings of the 23rd Annual ACM Symposium on Parallelism in Algorithms and Architectures, San Jose, CA, USA, June 4-6, 2011 (Co-located with FCRC 2011)* (2011), R. Rajaraman and F. M. auf der Heide, Eds., ACM, pp. 85–94.

[25] LINIAL, N. Distributive graph algorithms-global solutions from local data. In *28th Annual Symposium on Foundations of Computer Science, Los Angeles, California, USA, 27-29 October 1987* (1987), IEEE Computer Society, pp. 331–335.

[26] LINIAL, N. Locality in distributed graph algorithms. *SIAM J. Comput. 21*, 1 (1992), 193–201.

[27] NARKHEDE, N., SHAPIRA, G., AND PALINO, T. *Kafka – The Definitive Guide: Real-time data and stream processing at scale*. O Reilly, 2017.

[28] OWEN, S., ANIL, R., DUNNING, T., AND FRIEDMAN, E. *Mahout in Action*. Manning Publications, 2011.

[29] PANDURANGAN, G., ROBINSON, P., AND SCQUIZZATO, M. Fast distributed algorithms for connectivity and mst in large graphs. *ACM Trans. Parallel Comput. 5*, 1 (June 2018).

[30] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., ET AL. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* (2019), vol. 32.

[31] PELEG, D. *Distributed computing: a locality-sensitive approach*. SIAM, Philadelphia, 2000.

[32] REED, D., AND DONGARRA, J. Exascale computing and big data. *Communication of the ACM 58*, 7 (2015), 56–68.

[33] VALIANT, L. G. A bridging model for parallel computation. *Commun. ACM 33*, 8 (1990), 103–111.